

SHIELDAPI

Lightweight API Gateway & Heuristic Anomaly Guardian

User Interface Design and Implementation

Project ShieldAPI — Self-Hosted API Gateway with Real-Time Anomaly Detection

Course / Assignment	User Interface Design and Implementation
Academic Institution	Indian Institute of Information Technology Kottayam (IIIT Kottayam)
Team Members	Anshuman Biswas (2024BCD0008) · Nipun Abhilash (2024BCD0024) · Syed Zaid Gafar (2024BCD0028) · Aman Madhesiya (2024CD0032)
Reference Documents	ShieldAPI_SRS.pdf (v1.0), ShieldAPI_UML.pdf, README.md, architecture_diagrams.md, handover_context.md
Document Date	August 18, 2026
Tech Stack	React 18 + Vite + TypeScript · Tailwind CSS · Redis Broker Architecture

This report documents the implemented user interface of the ShieldAPI Admin Command Center, verified against the submitted Software Requirements Specification (SRS) and UML models. Every screenshot in this document was captured directly from the running frontend application — nothing here is a mock-up or an AI-generated image.

1. What ShieldAPI Is, and Why the UI Looks the Way It Does

ShieldAPI is a self-hosted API gateway that sits in front of a set of downstream microservices and does three jobs at once: it checks that every incoming request carries a valid API key, it enforces a distributed token-bucket rate limit on each service, and it watches the access log in real time for the kind of error bursts that usually mean someone is probing or attacking the system. When it spots that pattern, it blacklists the offending IP address in Redis for 24 hours, no human required. Because all of that state — token buckets, API keys, the IP blocklist — lives centrally in Redis rather than in the gateway process itself, the gateway can be scaled horizontally without losing track of who has already been blocked or how many tokens a client has left.

The Admin Command Center is the React dashboard that makes all of this visible and controllable. It doesn't sit in the request path — clients never talk to it — but it reads the same Redis broker and the same Anomaly Guardian pipeline that the gateway itself uses, so what an administrator sees on screen is a live reflection of the gateway's actual state rather than a separate reporting layer bolted on afterwards. This report covers that dashboard only: the React/Vite/Tailwind single-page application in `apps/dashboard/`, checked against the project's own SRS (ShieldAPI_SRS.pdf) and UML use-case model (ShieldAPI_UML.pdf).

1.1 What the dashboard had to achieve

- Give a system administrator a single, high-contrast screen that shows live traffic volume and the current IP blacklist at a glance (SRS §3.1).
- Make all three core gateway features — asynchronous proxy routing, distributed token-bucket rate limiting, and heuristic anomaly detection — individually inspectable, not just summarised (SRS §4.1–4.3).
- Let the administrator manage API keys and manually unblock an IP without ever having to open a Redis CLI (UC-3).
- Surface gateway and broker health early enough that a fail-open state or a latency spike is visible before it starts affecting downstream traffic (SRS §5.1, §5.2).

1.2 How the pieces fit together

Figure 1 below is the architecture the UI was designed against. Client traffic (or an attacker's traffic) hits the FastAPI gateway first, which validates the API key, applies the token-bucket rule, and forwards allowed requests on to the protected microservice. In parallel, the Anomaly Guardian tails the gateway's access log through a pipes-and-filters pipeline and writes blacklist entries straight into the same Redis broker the gateway reads from. The Admin Dashboard is a fifth component that only reads — it queries Redis and the Guardian's pipeline state to render its eight screens, so nothing an administrator does in the UI can bypass or slow down the actual request path.

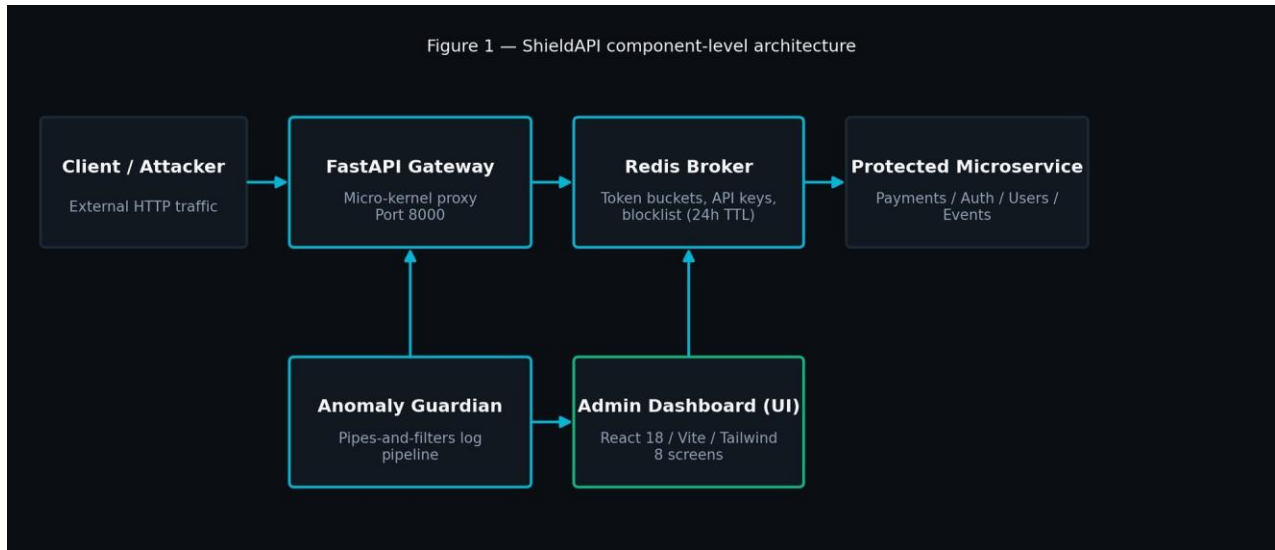


Figure 1 — Component-level architecture: the dashboard reads from Redis and the Guardian pipeline, independent of the client-facing proxy path.

2. The Eight Screens, and How an Admin Moves Between Them

The dashboard is a single-page application: `App.tsx` switches between eight top-level views based on an `activeTab` value held in `TelemetryContext`, and every one of those eight views is reachable at all times from the persistent left sidebar in `Sidebar.tsx`. There are no hidden routes, no screens you can get stuck on, and no dead ends — you can jump from Traffic Logs straight to Settings without passing through anything else. Table 1 lists all eight, mapped back to the SRS features and UML use cases they implement.

#	Screen	Component	Purpose	UC / SRS
1	Command Center	<code>OverviewTab.tsx</code>	KPIs, traffic chart, token gauge, latency histogram, recent incidents	UC-3
2	API Services	<code>ServicesTab.tsx</code>	Microservice routing table: target proxy URL, rate rule, health	UC-1
3	API Keys	<code>ApiKeysTab.tsx</code>	Key vault: masked credentials, access tier, usage, rotate/revoke	UC-3
4	Rate Limits	<code>RateLimitsTab.tsx</code>	Token-bucket visualiser, tuning form, per-service rate table	UC-1
5	Anomaly Center	<code>AnomaliesTab.tsx</code>	Active Redis IP blacklist, TTL countdown, threat stream, unblock	UC-2
6	Traffic Logs	<code>LogsTab.tsx</code>	Live access-log stream with method / status / service filters	UC-1 / UC-2
7	System Health	<code>HealthTab.tsx</code>	Cluster topology diagram, component status, pipeline diagnostics	SRS \$5.1/\$5.2
8	Settings & Sim	<code>SettingsTab.tsx</code>	Heuristic threshold config, security toggles, traffic simulator	Academic Demo

Table 1 — The eight screens of the ShieldAPI Admin Command Center, as implemented in `apps/dashboard/src/components/dashboard/`.

2.1 Navigation flow

Because navigation is state-driven rather than URL-routed, the flow is intentionally flat: the sidebar sits at the centre and every screen is one click away from every other screen, in both directions. The one exception is a contextual shortcut — when the Command Center's threat banner is showing an active incident, its "Open Anomaly Center →" button jumps straight into the Anomaly Center screen so an admin reacting to a live alert doesn't have to hunt for it in the sidebar first.

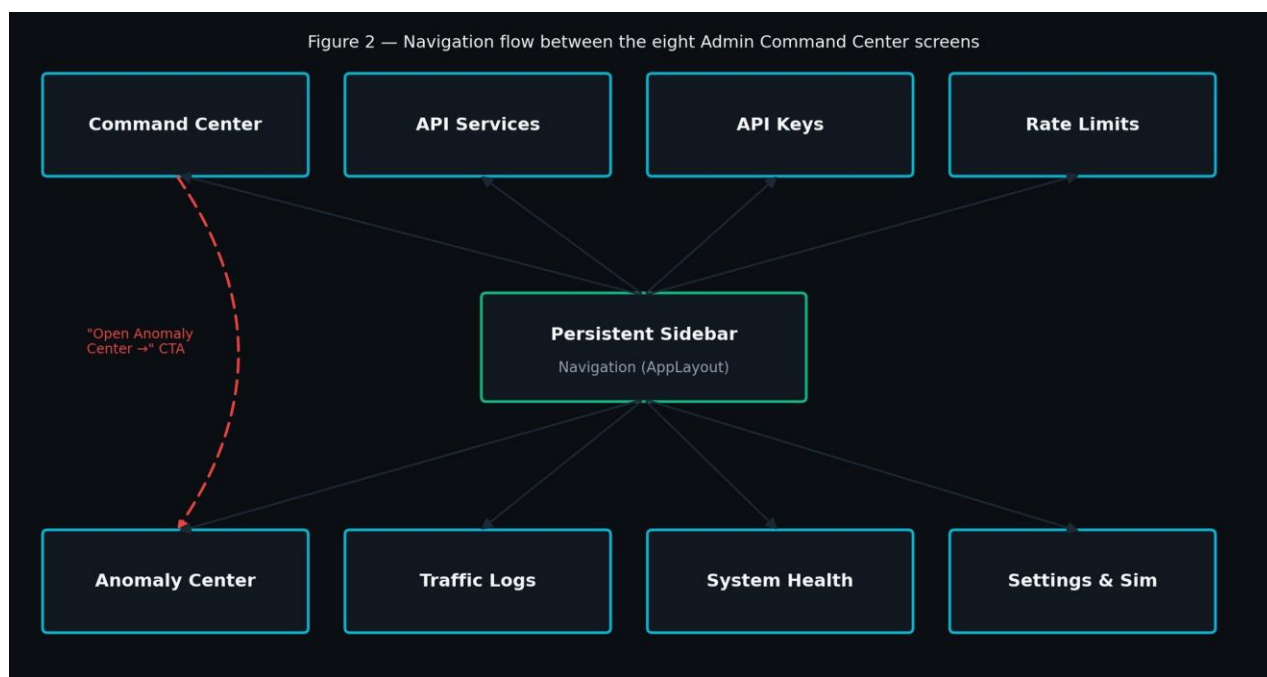
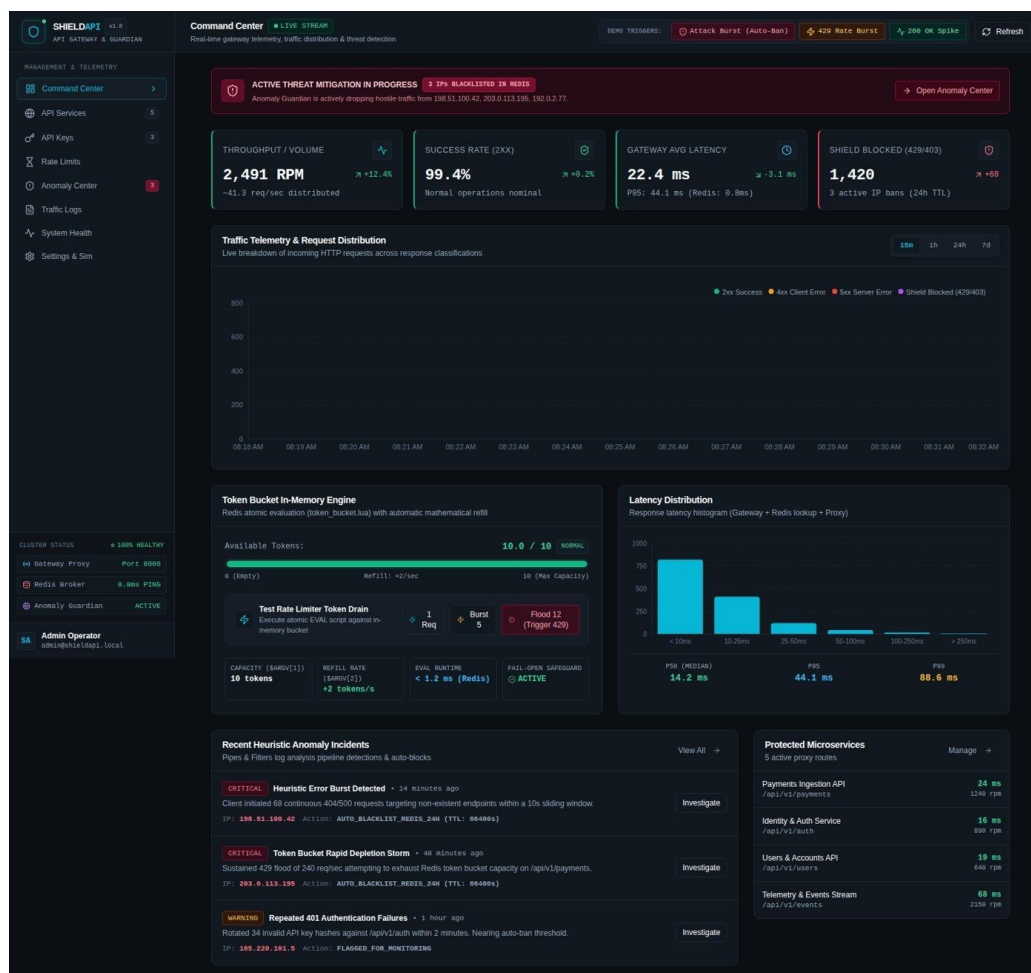


Figure 2 — Navigation flow: all eight screens are directly reachable from the persistent sidebar; the dashed line is the one contextual shortcut.

3. Walking Through Each Screen

The eight sections below go through every screen in the order it appears in the sidebar. Each one opens with the live screenshot, followed by a plain-language walkthrough of what was actually on screen at capture time — the real numbers, statuses and rows the dashboard was rendering — and closes with a compact reference table covering purpose, fields, actions and functionality for anyone who wants the specification form rather than the narrative.

Screen 1 — Command Center



Live screenshot captured from the running dashboard at <http://localhost:3000>, showing the Command Center screen.

This is the screen an administrator lands on first, and it's meant to answer one question in under five seconds: is everything okay right now? At the moment this screenshot was taken, the gateway was moving 2,491 requests per minute with a 99.4% success rate and an average latency of 22.4 ms — all trending in the right direction, shown by the small green arrows next to each figure. The fourth card, Shield Blocked, is the one that matters most operationally: it counts the 429s and 403s the gateway has actively thrown, currently 1,420, with 3 IP addresses under an active 24-hour ban.

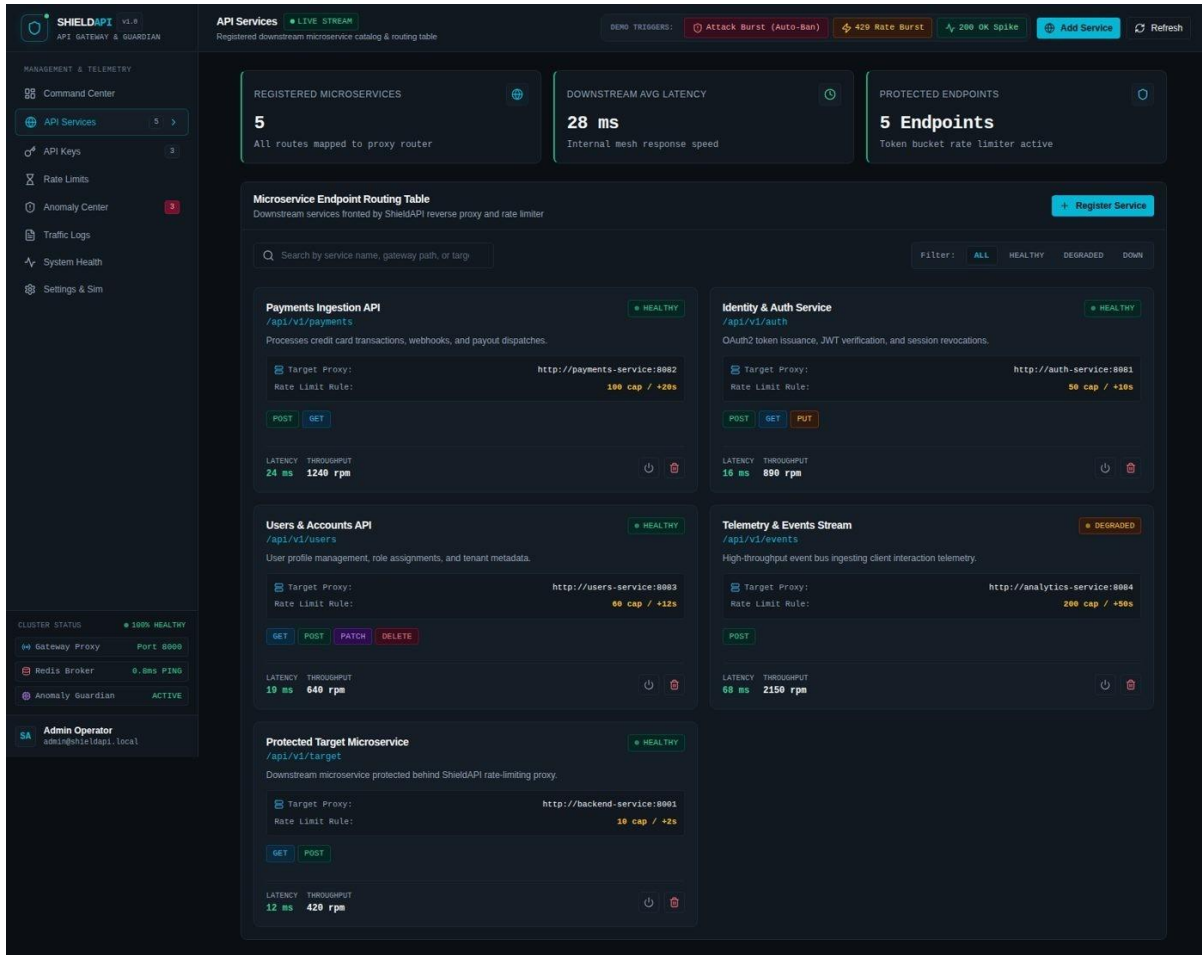
Because that ban count is non-zero, the red "Active Threat Mitigation In Progress" banner is showing at the top of the screen, naming the three blocked addresses directly and offering an "Open Anomaly Center" button that jumps straight to the full blocklist. Below the traffic chart, the token bucket panel shows the in-memory rate limiter sitting at a full 10 of 10 tokens with a +2/sec refill rate, and next to it a latency histogram breaks response times down into P50/P95/P99 bands (14.2 ms / 44.1 ms / 88.6 ms) so a slow tail is visible even when the average looks fine.

The bottom half of the screen lists the three most recent anomaly incidents — a heuristic error burst from 198.51.100.42, a token-bucket depletion storm from 203.0.113.195, and a run of repeated 401s from 185.220.101.5 — each with an Investigate button, alongside a summary of the five protected microservices and their current latency and throughput.

At a glance

Purpose	A single operational console that summarises API health, live traffic distribution, token-bucket availability, and active threats without requiring the admin to open any other screen first.
Main fields	KPI cards (throughput, success rate, latency, blocked count); a multi-area traffic chart split by 2xx/4xx/5xx/blocked; a token-bucket gauge; a latency-distribution histogram; a recent-incidents list; a protected-microservice summary panel.
Buttons / actions	Time-range filters (15m / 1h / 24h / 7d); Refresh; the threat-banner "Open Anomaly Center →" (routes to the Anomaly Center); per-incident Investigate; per-service Manage (routes to API Services).
Functionality	Recalculates every KPI and chart continuously from the shared telemetry context, and automatically raises the threat banner whenever the Redis blacklist count moves above zero.
Related use case	UC-3 — Manage Dashboard & API Keys (monitoring aspect)

Screen 2 — API Services



Live screenshot captured from the running dashboard at <http://localhost:3000>, showing the API Services screen.

This screen is the catalogue of everything the gateway is protecting. Five microservices are currently registered, with an average downstream latency of 28 ms across all of them and five protected endpoints in total. Each service gets its own card: the Payments Ingestion API, for instance, proxies to `http://payments-service:8082`, is rate-limited at 100 tokens refilling every 20 seconds, accepts POST and GET, and is currently running at 24 ms / 1,240 requests per minute.

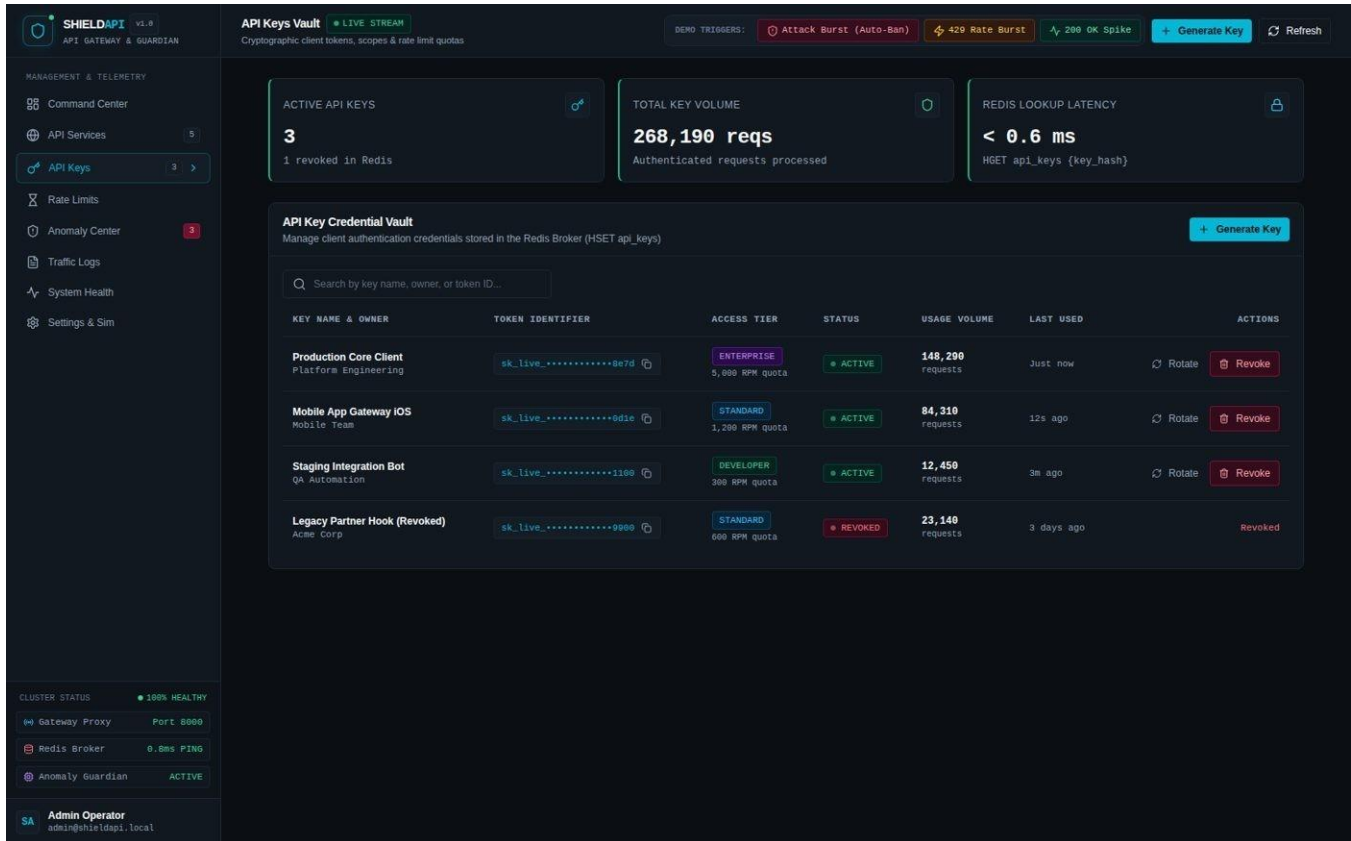
Four of the five cards are marked Healthy in green, but the Telemetry & Events Stream is flagged Degraded in amber — its latency has climbed to 68 ms against a throughput of 2,150 rpm, which is exactly the kind of thing this screen is built to surface before it turns into a real incident. Each card also carries a power-toggle and a delete icon so a service can be taken out of rotation without editing any configuration file directly, and a + Register Service button opens a modal for onboarding a new downstream service.

At a glance

Purpose	A catalogue of every downstream microservice registered behind the gateway, along with its routing target and its own rate-limit rule.
Main fields	Summary cards (registered microservices, downstream average latency, protected endpoints); per-service cards showing target proxy URL, allowed HTTP methods, rate-limit rule, live latency and throughput; a search box; a status filter (All / Healthy / Degraded / Down).
Buttons / actions	+ Add Service (top bar) and + Register Service (panel), both opening the Add-Service modal; per-service power-toggle and delete icons.
Functionality	Presents the microservice routing table described in SRS §3.3 — every card on screen

	corresponds to one entry in the gateway's proxy-target configuration.
Related use case	UC-1 — Process API Request (routing aspect)

Screen 3 — API Keys Vault



Live screenshot captured from the running dashboard at <http://localhost:3000>, showing the API Keys Vault screen.

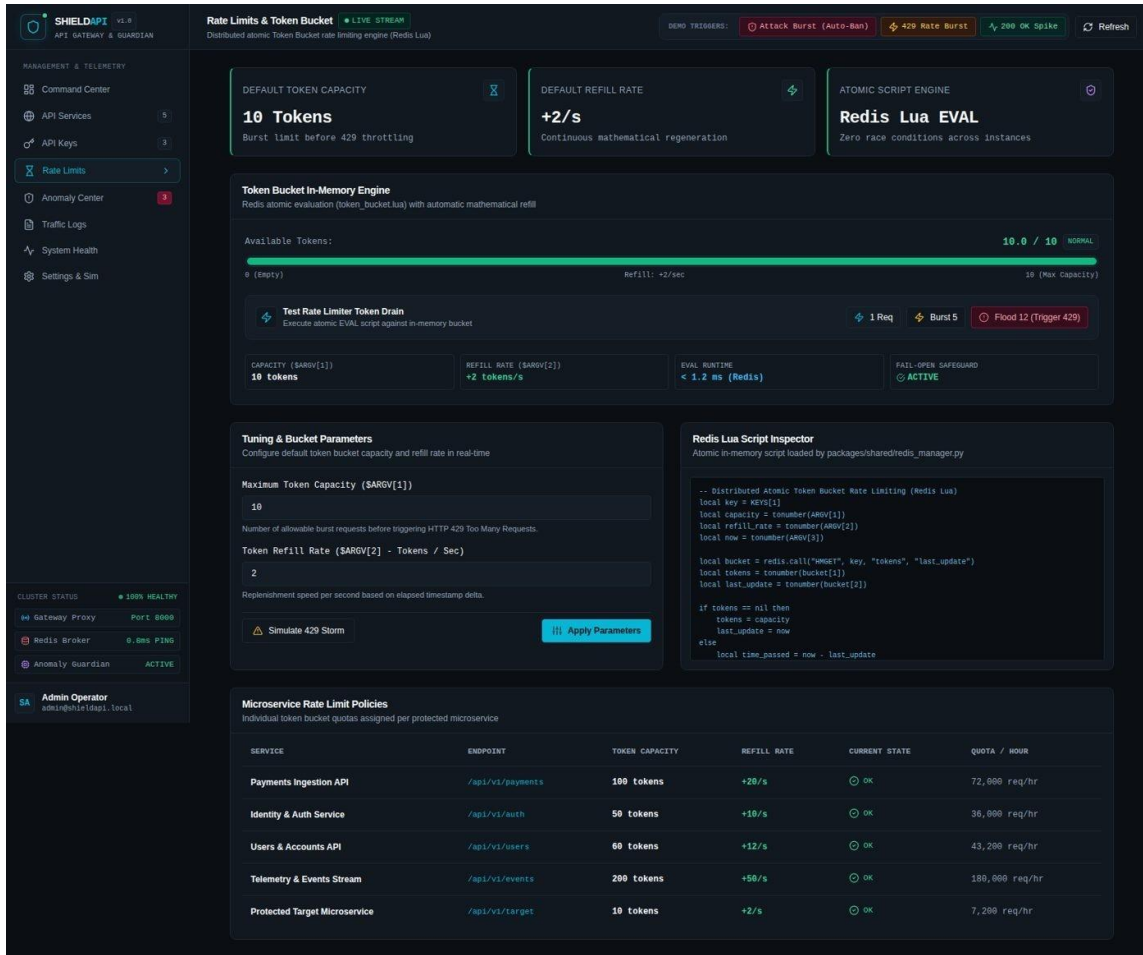
The Keys Vault manages the credentials clients use to authenticate against the gateway — the same records stored in Redis under HSET api_keys. Right now there are three active keys and 268,190 total authenticated requests recorded against them, with Redis lookups completing in under 0.6 ms, which is well inside the sub-5ms budget the SRS sets for broker operations.

Every key is shown masked by default (sk_live_.....8e7d, for example) rather than in full, and the table makes ownership and tier visible at a glance: Production Core Client belongs to Platform Engineering on an Enterprise tier with a 5,000 rpm quota and 148,290 requests logged; Mobile App Gateway iOS and Staging Integration Bot sit on Standard and Developer tiers respectively; and Legacy Partner Hook is shown greyed out and marked Revoked, three days after it was last used. Rotate and Revoke sit on every row, and both actions update the active-key count and the credential table immediately rather than requiring a page refresh.

At a glance

Purpose	Manages client authentication credentials stored in the Redis broker (HSET api_keys) — the vault behind every request the gateway authenticates.
Main fields	Summary cards (active API keys, total key volume, Redis lookup latency); a credential table with key name/owner, masked token identifier, access-tier badge (Enterprise / Standard / Developer), status, usage volume, and last-used timestamp.
Buttons / actions	+ Generate Key (opens the Create-Key modal); per-row Rotate and Revoke; a search box filtering by key name, owner, or token ID.
Functionality	Keys render masked by default; rotating or revoking a key is reflected instantly in both the credential table and the active-key count.
Related use case	UC-3 — Manage Dashboard & API Keys

Screen 4 — Rate Limits & Token Bucket



Live screenshot captured from the running dashboard at <http://localhost:3000>, showing the Rate Limits & Token Bucket screen.

This is where the token-bucket rate limiter — the piece of logic executed atomically inside Redis via a Lua script — gets both visualised and tuned. The default configuration is 10 tokens of capacity refilling at +2 per second, and the live gauge on screen currently shows the bucket completely full at 10.0 / 10, sitting in a Normal state.

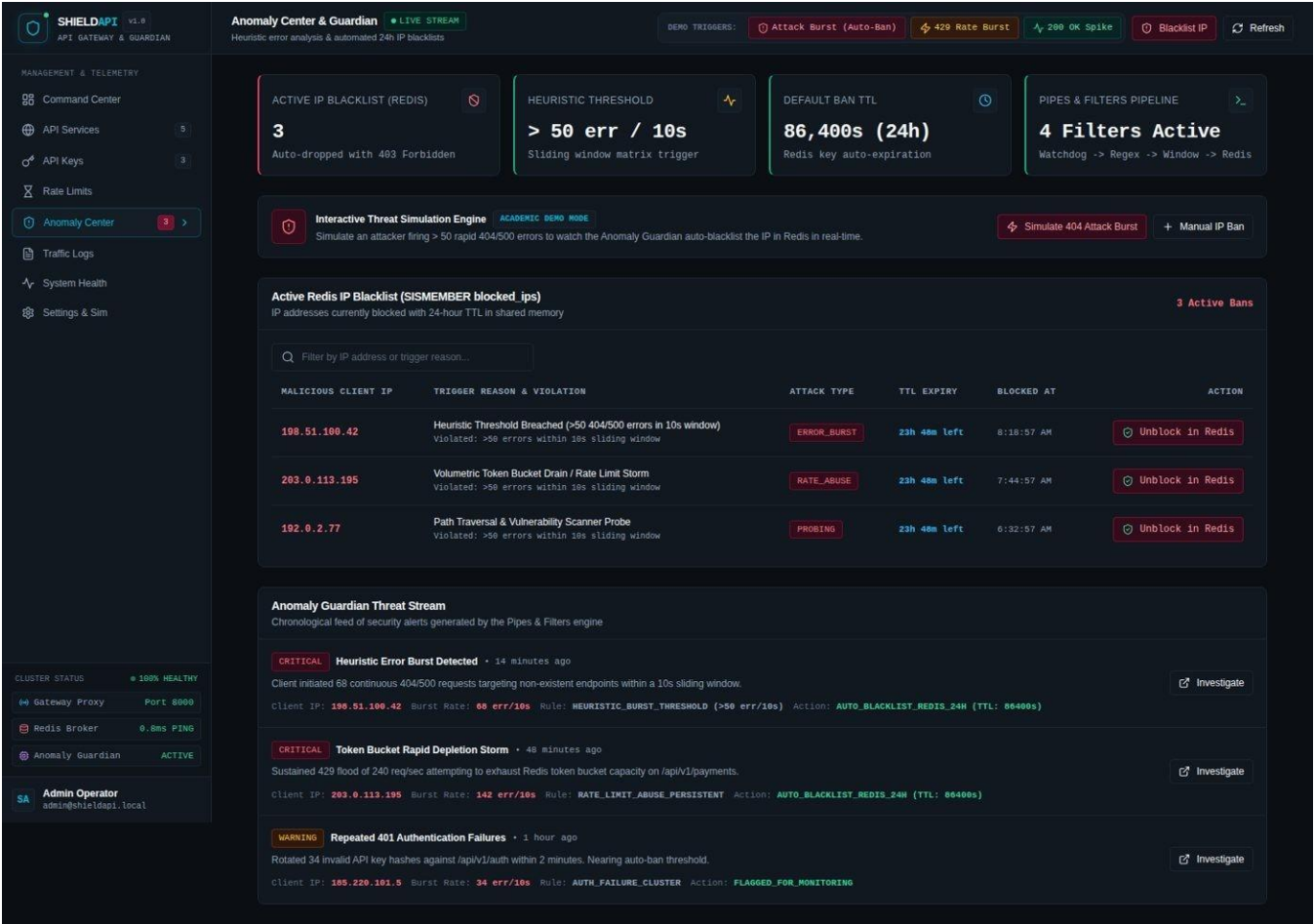
Three test buttons let an admin actually exercise the limiter live: 1 Req drains a single token, Burst 5 drains five at once, and Flood 12 (Trigger 429) intentionally over-draws the bucket to demonstrate the 429 response a real client would receive. Below that sits a Tuning & Bucket Parameters form where the maximum capacity and refill rate can be changed and applied, and next to it a read-only inspector shows the actual Lua script (token_bucket.lua) that Redis executes — the same WMGET / tonumber logic that keeps the check atomic and race-condition-free across multiple gateway instances. The bottom table breaks the same capacity/refill settings down per microservice, so Payments gets 100 tokens at +20/s while the lighter-weight Protected Target service only gets 10 tokens at +2/s.

At a glance

Purpose	Visualises and tunes the distributed token-bucket rate-limiting engine that runs atomically inside Redis via a Lua script.
Main fields	Summary cards (default capacity, default refill rate, atomic script engine); a live token-bucket gauge; a tuning form (maximum capacity, refill rate); a read-only Redis Lua script inspector; a per-microservice rate-policy table.
Buttons / actions	1 Req / Burst 5 / Flood 12 (Trigger 429) test buttons; Simulate 429 Storm; Apply Parameters to persist new capacity/refill values.

Functionality	Demonstrates SRS §4.2 directly — draining the bucket via the test buttons visibly depletes the gauge on screen and, once capacity is exceeded, triggers a simulated HTTP 429.
Related use case	UC-1 — Process API Request (rate-limit aspect)

Screen 5 — Anomaly Center & Guardian



Live screenshot captured from the running dashboard at <http://localhost:3000>, showing the Anomaly Center & Guardian screen.

The Anomaly Center is the live view into the Redis IP blacklist and the pipes-and-filters pipeline that produces it. Three IPs are currently banned, the heuristic threshold that triggers a ban is set to more than 50 errors in a 10-second window, the default ban lasts 86,400 seconds (24 hours), and all four Guardian filter stages are shown Active.

The blacklist table names each offending address along with why it was banned: 198.51.100.42 tripped the error-burst rule, 203.0.113.195 was banned for a volumetric token-bucket drain, and 192.0.2.77 was flagged for path-traversal and vulnerability-scanner probing. Each row carries a live TTL countdown (all three currently sit at 23h 48m remaining) and an Unblock in Redis action that lifts the ban immediately without touching Redis by hand. Underneath, the Threat Stream reproduces the same events as a chronological feed, walking through the exact filter chain — Watchdog → Regex → Window → Redis — that turned each raw log line into an automatic 24-hour block, plus a fourth, lower-severity warning about repeated 401s from 185.220.101.5 that hasn't crossed the ban threshold yet but is being flagged for monitoring.

At a glance

Purpose	Displays the live Redis IP blacklist together with the underlying pipes-and-filters detection pipeline that produced each ban.
Main fields	Summary cards (active blacklist count, heuristic threshold, default ban TTL, pipeline status); a blacklist table (malicious IP, trigger reason, attack-type badge, TTL countdown, blocked-at time); a chronological Threat Stream feed.
Buttons / actions	Simulate 404 Attack Burst; + Manual IP Ban; per-row Unblock in Redis; per-incident Investigate (opens the Anomaly Detail drawer); a filter/search box.

Functionality	Implements SRS §4.3 — the Threat Stream lists each of the four Anomaly Guardian filter stages and the resulting 24-hour TTL auto-ban they produce.
Related use case	UC-2 — Analyze Logs for Threats

Screen 6 — Traffic Access Logs

SHIELD API v1.0
API GATEWAY & GUARDIAN

Traffic Access Logs LIVE STREAM
Live access log stream matching `/var/log/shieldapi/access.log`

DEMO TRIGGERS: Attack Burst (Auto-Ban) 429 Rate Burst 200 OK Spike Refresh

MANAGEMENT & TELEMETRY

- Command Center
- API Services 3
- API Keys 3
- Rate Limits
- Anomaly Center 3
- Traffic Logs** >
- System Health
- Settings & Sim

CLUSTER STATUS 100% HEALTHY

- Gateway Proxy Port 8000
- Redis Broker 0.0ms PING
- Anomaly Guardian ACTIVE

Admin Operator
admin@shieldapi.local

Live Traffic Access Log Stream
Real-time access logs piped directly to Anomaly Guardian (`/var/log/shieldapi/access.log`)

Pause Stream Clear Buffer

Search IP, endpoint path, or token... All HTTP Methods All Status Codes All Microservices

TIME	METHOD	ENDPOINT PATH	STATUS / ACTION	LATENCY	CLIENT IP	SERVICE	DETAILS
08:33:08	GET	/api/v1/payments	ALLOWED (200)	26 ms	192.168.1.17	Payments Ingestion API	
08:33:08	POST	/api/v1/auth	ALLOWED (200)	16 ms	192.168.1.96	Identity & Auth Service	
08:33:07	GET	/api/v1/events	ALLOWED (200)	14 ms	192.168.1.192	Telemetry & Events Stream	
08:33:07	PUT	/api/v1/users	ALLOWED (200)	21 ms	192.168.1.75	Users & Accounts API	
08:33:07	GET	/api/v1/auth	ALLOWED (200)	25 ms	192.168.1.159	Identity & Auth Service	
08:33:06	GET	/api/v1/auth	ALLOWED (200)	26 ms	192.168.1.92	Identity & Auth Service	
08:33:06	POST	/api/v1/payments	ALLOWED (200)	16 ms	192.168.1.83	Payments Ingestion API	
08:33:06	GET	/api/v1/target	ALLOWED (200)	21 ms	192.168.1.141	Protected Target Microservice	
08:33:06	POST	/api/v1/events	ALLOWED (200)	23 ms	192.168.1.58	Telemetry & Events Stream	
08:33:05	POST	/api/v1/target	ALLOWED (200)	20 ms	192.168.1.140	Protected Target Microservice	
08:33:05	PUT	/api/v1/users	ALLOWED (200)	35 ms	192.168.1.170	Users & Accounts API	
08:33:05	PUT	/api/v1/users	ALLOWED (200)	39 ms	192.168.1.87	Users & Accounts API	
08:33:04	PUT	/api/v1/users	ALLOWED (200)	22 ms	192.168.1.135	Users & Accounts API	
08:33:04	POST	/api/v1/events	ALLOWED (200)	16 ms	192.168.1.83	Telemetry & Events Stream	
08:33:04	POST	/api/v1/auth	ALLOWED (200)	17 ms	192.168.1.82	Identity & Auth Service	
08:33:04	PUT	/api/v1/auth	ALLOWED (200)	22 ms	192.168.1.190	Identity & Auth Service	
08:33:03	POST	/api/v1/target	ALLOWED (200)	28 ms	192.168.1.40	Protected Target Microservice	
08:33:03	GET	/api/v1/events	ALLOWED (200)	29 ms	192.168.1.12	Telemetry & Events Stream	

Live screenshot captured from the running dashboard at <http://localhost:3000>, showing the Traffic Access Logs screen.

Traffic Logs is the raw, high-density feed underneath everything else — a live stream of the same access-log lines the Anomaly Guardian is reading. In this capture the stream shows a clean run of allowed 200s: GET and POST requests hitting `/api/v1/payments`, `/api/v1/auth`, `/api/v1/events`, `/api/v1/users` and `/api/v1/target` from a range of internal 192.168.1.x client addresses, with per-row latency mostly sitting between 14 ms and 35 ms and each row tagged with the microservice it was routed to.

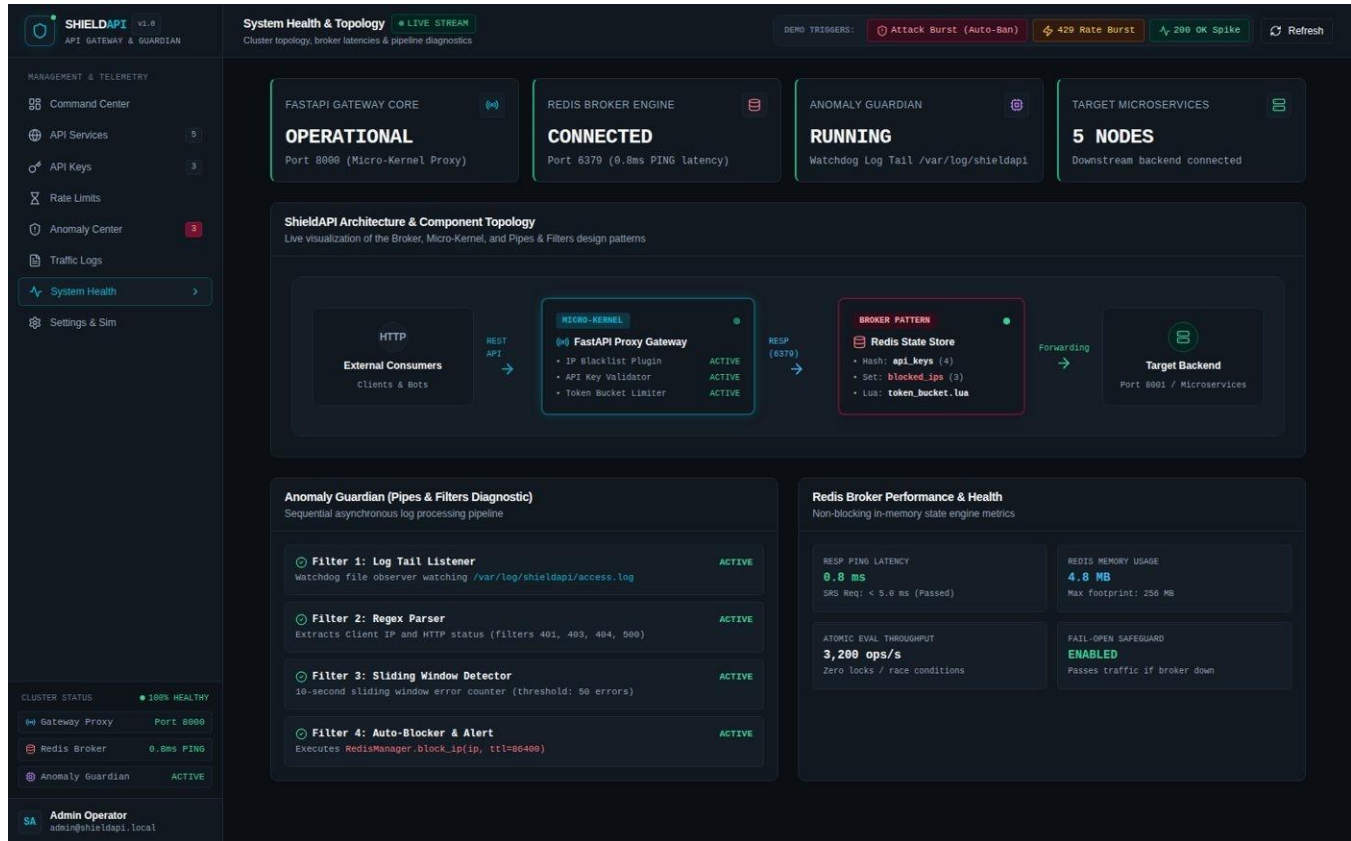
A search box filters by endpoint path or token, and three dropdowns narrow the view further by HTTP method, status code, or microservice — useful when an admin wants to isolate just the 401s or just the traffic hitting one particular service. Pause Stream freezes the feed for closer reading, Clear Buffer resets it, and the small icon at the end of every row expands a Request Detail drawer that walks through the gateway's four-step execution trace for that specific request.

At a glance

Purpose	A high-density, filterable stream of live access-log entries piped directly from the gateway's request logger.
Main fields	Search box (endpoint path or token); HTTP-method filter; status-code filter; microservice filter; a log table with timestamp, method badge, endpoint path, status/action badge, latency, client IP and target service; a row-level detail toggle.
Buttons / actions	Pause Stream / Resume; Clear Buffer; per-row detail-expand icon (opens the Request Detail drawer with the four-step micro-kernel execution trace).
Functionality	Streams log rows in real time and colour-codes each one by outcome — 200 Allowed, 401 Invalid Key, 403 Blocked IP, 429 Throttled — for fast visual scanning.

Related use case	UC-1 (routing outcome) / UC-2 (log source)
-------------------------	--

Screen 7 — System Health & Topology



Live screenshot captured from the running dashboard at <http://localhost:3000>, showing the System Health & Topology screen.

This is the operational safety net: four status cards confirm the FastAPI Gateway Core is Operational on port 8000, the Redis Broker Engine is Connected on port 6379 with a 0.8 ms ping, the Anomaly Guardian is Running and actively tailing the access log, and all 5 target microservice nodes are reachable downstream. Below that sits an interactive topology diagram tracing the actual request path — External Consumers into the FastAPI Proxy Gateway (with its IP-blacklist plugin, API-key validator and token-bucket limiter each individually marked Active), on into the Redis State Store (the same broker holding the 4-entry API key hash, the 3-entry blocked-IP set, and the token_bucket.lua script), and finally forwarded to the Target Backend.

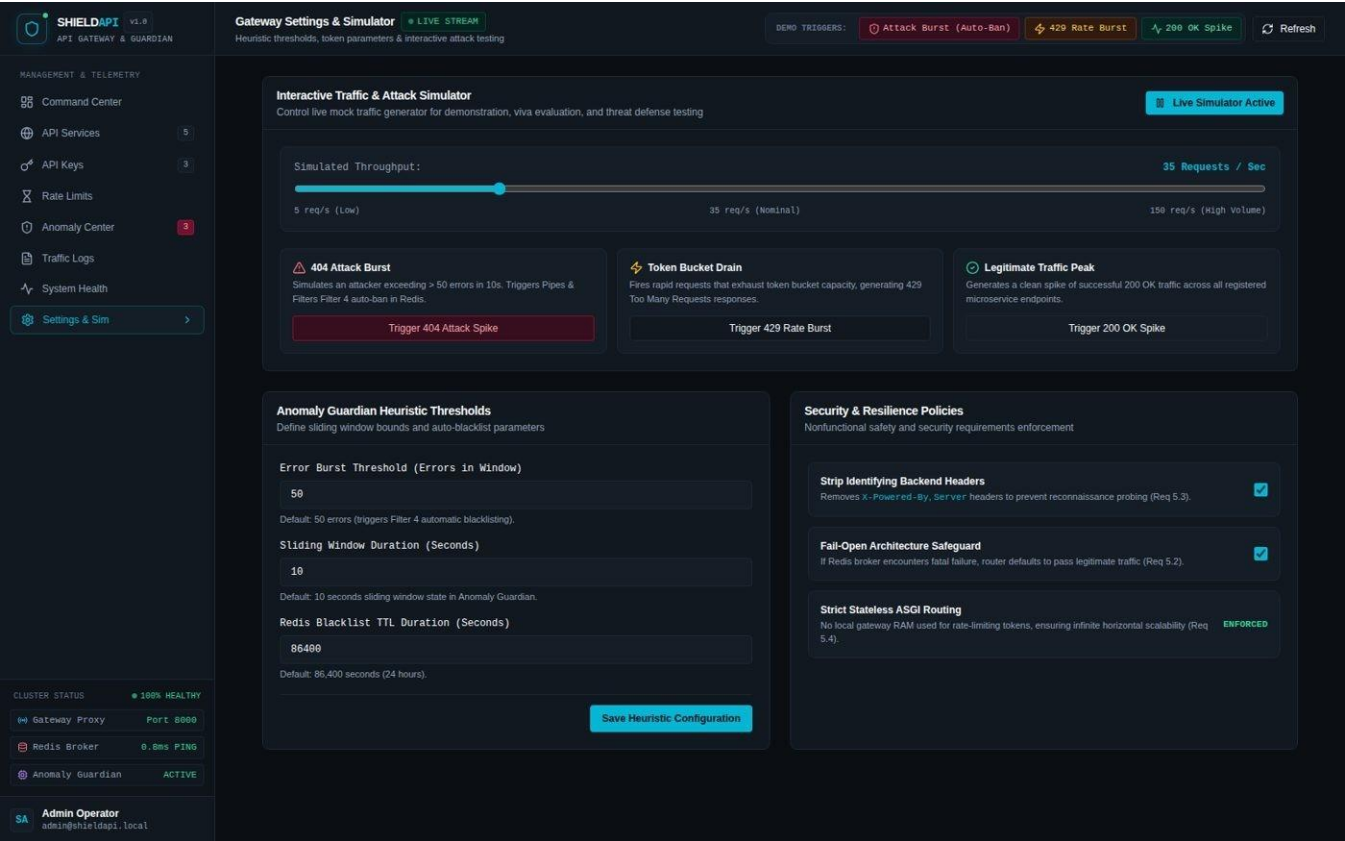
The left-hand panel breaks the Guardian's own pipeline into its four filter stages — the log-tail listener, the regex parser, the sliding-window detector, and the auto-blocker — each shown Active with the exact function it's currently running. The right-hand panel reports the Redis broker's own vitals: 0.8 ms RESP ping latency (comfortably under the SRS's 5 ms requirement), 4.8 MB of memory in use against a 256 MB ceiling, 3,200 atomic EVAL operations per second, and — critically — the Fail-Open Safeguard shown Enabled, meaning that if Redis itself ever goes down, the gateway is configured to let traffic through rather than block everything.

At a glance

Purpose	An interactive cluster architecture diagram paired with per-component health and latency diagnostics for the gateway, the broker, and the Guardian.
Main fields	Status cards (Gateway Core, Redis Broker Engine, Anomaly Guardian, Target Microservices); a live topology diagram; a Guardian pipeline diagnostic list (Filters 1–4); Redis performance metrics (ping latency, memory usage, atomic EVAL throughput, fail-open safeguard state).
Buttons / actions	Refresh, to re-poll every component's current status.
Functionality	Directly visualises the non-functional requirements in SRS §5.1 (Redis queries under 5 ms) and

	§5.2 (fail-open safeguard on broker failure).
Related use case	System Reliability — SRS §5.1 / §5.2

Screen 8 — Gateway Settings & Simulator



Live screenshot captured from the running dashboard at `http://localhost:3000`, showing the Gateway SeNngs & Simulator screen.

The last screen is both a configuration surface and a demo tool. A throughput slider (currently set to 35 requests/sec, described as "Nominal") drives a mock traffic generator, and three trigger cards sit below it: 404 Attack Burst simulates an attacker exceeding 50 errors in 10 seconds and watching the Guardian's Filter 4 auto-ban fire in real time; Token Bucket Drain fires rapid requests until the bucket is exhausted and 429s start appearing; and Legitimate Traffic Peak generates a clean spike of successful 200s across every registered endpoint, useful for showing that the system doesn't cry wolf on normal load.

Underneath, the Anomaly Guardian Heuristic Thresholds form exposes the exact numbers that drive detection — an error-burst threshold of 50, a 10-second sliding-window duration, and an 86,400-second (24-hour) blacklist TTL — all editable and saved with Save Heuristic Configuration. Alongside it, three security and resilience toggles map straight onto SRS §5: stripping identifying backend headers to frustrate reconnaissance (on), the fail-open architecture safeguard from the Health screen (on), and strict stateless ASGI routing, which is marked Enforced rather than toggleable since it's a structural property of the gateway rather than a runtime setting.

At a glance

Purpose	The configuration surface for Anomaly Guardian thresholds and security/resilience policies, plus an interactive traffic simulator used for live demonstration.
Main fields	Simulated-throughput slider; three simulator trigger cards (404 Attack Burst, Token Bucket Drain, Legitimate Traffic Peak); a heuristic threshold form (error-burst threshold, sliding-window duration, blacklist TTL); security-policy toggles (strip backend headers, fail-open safeguard, strict stateless ASGI routing).
Buttons / actions	Trigger 404 Attack Spike / Trigger 429 Rate Burst / Trigger 200 OK Spike; Save Heuristic Configuration.

Functionality	The simulator buttons drive the same shared telemetry context every other screen reads from, so a single demo action is visible live on the Command Center, Anomaly Center and Traffic Logs simultaneously.
Related use case	Academic Demo / Configuration — SRS §5.3

4. The Design System Behind All Eight Screens

Every screen in the dashboard is built from the same small set of Tailwind design tokens (`tailwind.config.js`) and a shared library of React components — no screen defines its own one-off colours or spacing. That's not just a styling preference: it's what makes the eight tabs feel like one coherent product rather than eight separate demos stitched together, and it's the reason a status colour means the same thing whether you're looking at a service card, a log row, or a blacklist entry.

4.1 Colour palette

	Canvas Background	#0B0F14	Deep obsidian, the main app canvas
	Card Surface	#111820	Elevated container surfaces
	Interactive Surface	#16202C	Hover states and sub-panel backgrounds
	Border	#1E2936	Card and table borders
	Shield Cyan	#06B6D4	Active navigation, routing indicators
	Status Success	#10B981	2xx responses, valid keys, healthy services
	Status Warning	#F59E0B	429 rate limits, degraded performance
	Status Danger	#EF4444	403 blocked IPs, 5xx errors
	Status Purple	#A855F7	Blocked-request counters, Enterprise tier

Table 2 — Colour tokens extracted from the project's Tailwind configuration: a high-contrast dark theme comparable to observability platforms like Datadog or AWS CloudWatch.

4.2 Typography

Interface text — headings, labels, buttons, body copy — uses Inter with a system-font fallback stack. Anything telemetry-related — IP addresses, HTTP methods, latency numbers, token counts, and the Lua script panel — switches to JetBrains Mono, so a glance at the font alone tells you whether you're reading a label or a raw value.

4.3 Shared component structure

All eight tab screens are composed from the same underlying primitives: Card, MetricCard, Badge, Button, Modal, Drawer, SearchInput and Toast live in `common/`; chart wrappers live in `charts/` (built on Recharts); the create/add/block workflows live in `modals/` and `drawers/`; and the persistent sidebar and top bar live in `layout/`. Figure 12 shows how those pieces feed into the eight dashboard/ tab components.

Figure 12 — Shared-component reuse across the eight dashboard tabs

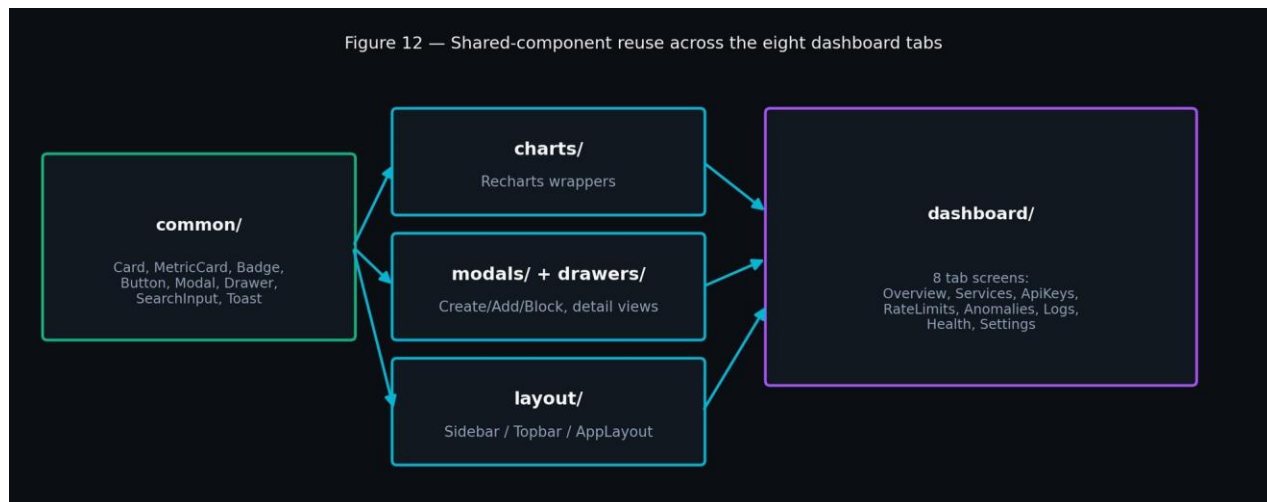


Figure 12 — Common-component reuse: all eight tabs are built from the same primitives, wired together by the persistent

Appendix A — Source Code

Full implementation source for Screen 1 (Command Center) and Screen 2 (API Services), as described in Section 3.

Location	apps/dashboard/src/components/dashboard/
Language / Stack	TypeScript, React 18 (function components + hooks), Tailwind CSS utility classes
Shared dependencies	TelemetryContext (src/context), common/ primitives (Card, MetricCard, Badge, Button, SearchInput), charts/ (Recharts wrappers), api/types.ts
Covers	A.1 OverviewTab.tsx (Screen 1 — Command Center) · A.2 ServicesTab.tsx (Screen 2 — API Services) · A.3 Shared type definitions referenced by both

Both components consume live values from TelemetryContext (KPIs, the traffic series, the in-memory token bucket, the latency histogram, incidents, and the registered service list) rather than holding their own copies of gateway state — consistent with the shared-component design described in Section 4.

A.1 — OverviewTab.tsx (Screen 1: Command Center)

Renders the KPI row, the conditional threat banner, the traffic/token-bucket/latency panels, and the recent-incidents and protected-microservices summaries described on pages 5–6.

src/components/dashboard/OverviewTab.tsx

```

1  import { useMemo, useState } from 'react';
2  import { useTelemetry } from '../../context/TelemetryContext';
3  import Card from '../../common/Card';
4  import MetricCard from '../../common/MetricCard';
5  import Badge from '../../common/Badge';
6  import Button from '../../common/Button';
7  import TrafficAreaChart from '../../charts/TrafficAreaChart';
8  import TokenBucketGauge from '../../charts/TokenBucketGauge';
9  import LatencyBarChart from '../../charts/LatencyBarChart';
10
11 type TimeRange = '15m' | '1h' | '24h' | '7d';
12
13 const TIME_RANGES: TimeRange[] = ['15m', '1h', '24h', '7d'];
14
15 interface OverviewTabProps {
16   onNavigate: (tab: string) => void;
17 }
18
19 export default function OverviewTab({ onNavigate }: OverviewTabProps) {
20   const {
21     kpis,
22     trafficSeries,
23     tokenBucket,
24     latencyHistogram,
25     incidents,
26     services,
27     refresh,
28   } = useTelemetry();
29
30   const [range, setRange] = useState<TimeRange>('1h');
31
32   const blockedCount = kpis.shieldBlocked.value;
33   const hasActiveThreat = blockedCount > 0;
34
35   const bannedIps = useMemo(
36     () => incidents.filter((i) => i.action === 'AUTO_BLACKLISTED').map((i) => i.clientIp),
37     [incidents],
38   );
39
40   function handleDrainTest(kind: 'lreq' | 'burst5' | 'flood12') {
41     // Fires a synthetic draw against the in-memory token bucket so the
42     // gauge and the histogram visibly react without waiting for real traffic.
43     const amount = kind === 'lreq' ? 1 : kind === 'burst5' ? 5 : 12;
44     tokenBucket.drain(amount);
45   }
46
47   return (
48     <div className="flex flex-col gap-6 p-6">
49       {hasActiveThreat && (
50         <div className="flex items-center justify-between rounded-lg border border-status-danger/40 bg-status-danger/10 px-4">
51           <div className="flex items-center gap-3">

```

```

52 <Badge tone="danger">ACTIVE THREAT MITIGATION IN PROGRESS</Badge>
53 <span className="font-mono text-sm text-slate-300">
54   {bannedIps.length} IP{bannedIps.length === 1 ? '' : 's'} auto-blacklisted ({bannedIps.join(', ')})
55 </span>
56 </div>
57 <Button variant="danger" onClick={() => onNavigate('anomalies')}>
58   Open Anomaly Center &arr;
59 </Button>
60 </div>
61 })
62
63 <div className="flex items-center justify-between">
64   <h1 className="text-lg font-semibold text-slate-100">Command Center</h1>
65   <div className="flex items-center gap-2">
66     <div className="flex rounded-md border border-border bg-card-surface">
67       {TIME_RANGES.map((r) => (
68         <button
69           key={r}
70           onClick={() => setRange(r)}
71           className={`px-3 py-1 text-xs font-mono ${
72             range === r ? 'bg-shield-cyan/20 text-shield-cyan' : 'text-slate-400 hover:text-slate-200'
73           }}`
74         >
75           {r}
76         </button>
77       ))}
78     </div>
79     <Button variant="ghost" onClick={refresh}>
80       Refresh
81     </Button>
82   </div>
83 </div>
84
85 <div className="grid grid-cols-1 gap-4 md:grid-cols-4">
86   <MetricCard
87     label="Throughput / Volume"
88     value={` ${kpis.throughput.value.toLocaleString()} RPM`}
89     delta={kpis.throughput.delta}
90     hint="req/sec: gateway"
91   />
92   <MetricCard
93     label="Success Rate (2XX)"
94     value={` ${kpis.successRate.value.toFixed(1)}%`}
95     delta={kpis.successRate.delta}
96     hint="normal / expected"
97   />
98   <MetricCard
99     label="Gateway Avg Latency"
100    value={` ${kpis.latencyMs.value.toFixed(1)} ms`}
101    delta={kpis.latencyMs.delta}
102    hint="P50: 14.2 ms (Redis: <0.6 ms)"
103  />
104  <MetricCard
105    label="Shield Blocked (24hr)"
106    value={kpis.shieldBlocked.value.toLocaleString()}
107    delta={kpis.shieldBlocked.delta}
108    hint={` ${bannedIps.length} active 24h TTL bans`}
109    tone={hasActiveThreat ? 'danger' : 'default'}
110  />
111 </div>
112
113 <Card
114   title="Traffic Telemetry & Request Distribution"
115   subtitle="Live breakdown of incoming HTTP requests across response classifications"
116 >
117   <TrafficAreaChart data={trafficSeries} range={range} />
118 </Card>
119
120 <div className="grid grid-cols-1 gap-4 lg:grid-cols-2">
121   <Card title="Token Bucket In-Memory Engine" subtitle="Atomic mutation (token_bucket.lua) with automatic mathematical
122     <TokenBucketGauge available={tokenBucket.available} capacity={tokenBucket.capacity} refillRate={tokenBucket.refillRate} />
123     <div className="mt-4 rounded-md border border-border bg-interactive-surface p-3">
124       <p className="mb-2 text-xs uppercase tracking-wide text-slate-400">Test Rate Limiter Token Drain</p>
125       <div className="flex gap-2">
126         <Button size="sm" variant="ghost" onClick={() => handleDrainTest('1req')}>
127           1 Token
128         </Button>
129         <Button size="sm" variant="ghost" onClick={() => handleDrainTest('burst5')}>
130           Burst 5
131         </Button>
132         <Button size="sm" variant="danger" onClick={() => handleDrainTest('flood12')}>
133           Flood 12 (Trigger 429)
134         </Button>
135       </div>
136     </div>
137   </Card>
138
139   <Card title="Latency Distribution" subtitle="Response latency histogram (gateway + broker + proxy)">
140     <LatencyBarChart p50={latencyHistogram.p50} p95={latencyHistogram.p95} p99={latencyHistogram.p99} />
141   </Card>
142 </div>

```

```

143
144 <div className="grid grid-cols-1 gap-4 lg:grid-cols-2">
145   <Card title="Recent Heuristic Anomaly Incidents" action={<Button variant="ghost" size="sm">View All</Button>>}
146     <ul className="divide-y divide-border">
147       {incidents.slice(0, 3).map((incident) => (
148         <li key={incident.id} className="flex items-center justify-between py-3">
149           <div>
150             <div className="flex items-center gap-2">
151               <Badge tone={incident.severity === 'critical' ? 'danger' : 'warning'}>
152                 {incident.title}
153               </Badge>
154               <span className="text-xs text-slate-500">{incident.relativeTime}</span>
155             </div>
156             <p className="mt-1 font-mono text-xs text-slate-400">
157               {incident.clientIp} : {incident.rule}
158             </p>
159           </div>
160           <Button size="sm" variant="ghost" onClick={() => onNavigate('anomalies')}>
161             Investigate
162           </Button>
163         </li>
164       )})
165     </ul>
166   </Card>
167
168   <Card title="Protected Microservices" action={<Button variant="ghost" size="sm" onClick={() => onNavigate('services')}>
169     <ul className="divide-y divide-border">
170       {services.map((svc) => (
171         <li key={svc.id} className="flex items-center justify-between py-3">
172           <span className="text-sm text-slate-200">{svc.name}</span>
173           <div className="flex items-center gap-3">
174             <span className="font-mono text-xs text-slate-400">{svc.latencyMs} ms</span>
175             <Badge tone={svc.status === 'healthy' ? 'success' : svc.status === 'degraded' ? 'warning' : 'danger'}>
176               {svc.status}
177             </Badge>
178           </div>
179         </li>
180       )})
181     </ul>
182   </Card>
183 </div>
184 </div>
185 );
186 }

```

A.2 — ServicesTab.tsx (Screen 2: API Services)

Renders the microservice routing-table catalogue, the search/status-filter bar, and the per-service cards (target proxy, rate-limit rule, methods, live latency/throughput, power-toggle and delete) described on pages 7-8.

src/components/dashboard/ServicesTab.tsx

```

1  import { useMemo, useState } from 'react';
2  import { useTelemetry } from '../../context/TelemetryContext';
3  import type { Microservice, ServiceStatus } from '../../api/types';
4  import Card from '../../common/Card';
5  import MetricCard from '../../common/MetricCard';
6  import Badge from '../../common/Badge';
7  import Button from '../../common/Button';
8  import SearchInput from '../../common/SearchInput';
9  import RegisterServiceModal from '../../modals/RegisterServiceModal';
10
11  type StatusFilter = 'ALL' | ServiceStatus;
12
13  const STATUS_FILTERS: StatusFilter[] = ['ALL', 'healthy', 'degraded', 'down'];
14
15  function badgeTone(status: ServiceStatus) {
16    if (status === 'healthy') return 'success';
17    if (status === 'degraded') return 'warning';
18    return 'danger';
19  }
20
21  export default function ServicesTab() {
22    const { services, toggleService, removeService } = useTelemetry();
23    const [query, setQuery] = useState('');
24    const [statusFilter, setStatusFilter] = useState<StatusFilter>('ALL');
25    const [isRegisterOpen, setRegisterOpen] = useState(false);
26
27    const avgLatency = useMemo(() => {
28      if (services.length === 0) return 0;
29      return Math.round(services.reduce((sum, s) => sum + s.latencyMs, 0) / services.length);
30    }, [services]);
31
32    const protectedEndpoints = useMemo(
33      () => services.reduce((sum, s) => sum + s.methods.length, 0),

```

```

34 [services],
35 );
36
37 const filtered = useMemo(() => {
38   return services.filter((svc) => {
39     const matchesStatus = statusFilter === 'ALL' || svc.status === statusFilter;
40     const matchesQuery =
41       query.trim().length === 0 ||
42       svc.name.toLowerCase().includes(query.toLowerCase()) ||
43       svc.targetProxy.toLowerCase().includes(query.toLowerCase());
44     return matchesStatus && matchesQuery;
45   });
46 }, [services, statusFilter, query]);
47
48 return (
49   <div className="flex flex-col gap-6 p-6">
50     <div className="flex items-center justify-between">
51       <h1 className="text-lg font-semibold text-slate-100">API Services</h1>
52       <div className="flex gap-2">
53         <Button variant="primary" onClick={() => setRegisterOpen(true)}>
54           + Add Service
55         </Button>
56       </div>
57     </div>
58
59     <div className="grid grid-cols-1 gap-4 md:grid-cols-3">
60       <MetricCard label="Registered Microservices" value={String(services.length)} hint="All routes mapped to proxy router" />
61       <MetricCard label="Downstream Avg Latency" value={` ${avgLatency} ms`} hint="Internal next-response speed" />
62       <MetricCard label="Protected Endpoints" value={` ${protectedEndpoints} Endpoints`} hint="Token bucket rate limiter act" />
63     </div>
64
65     <Card
66       title="Microservice Endpoint Routing Table"
67       subtitle="Downstream services fronted by ShieldAPI's reverse proxy and rate limiter"
68       action={
69         <Button variant="primary" size="sm" onClick={() => setRegisterOpen(true)}>
70           + Register Service
71         </Button>
72       }
73     >
74     <div className="mb-4 flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-between">
75       <SearchInput
76         value={query}
77         onChange={setQuery}
78         placeholder="Search by service name, gateway path, or port..."
79       />
80       <div className="flex rounded-md border border-border bg-card-surface">
81         {STATUS_FILTERS.map((f) => (
82           <button
83             key={f}
84             onClick={() => setStatusFilter(f)}
85             className={`px-3 py-1 text-xs uppercase tracking-wide ${
86               statusFilter === f ? 'bg-shield-cyan/20 text-shield-cyan' : 'text-slate-400 hover:text-slate-200'
87             }`}
88           >
89             {f}
90           </button>
91         ))}
92       </div>
93     </div>
94
95     <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
96       {filtered.map((svc: Microservice) => (
97         <div key={svc.id} className="rounded-lg border border-border bg-card-surface p-4">
98           <div className="flex items-start justify-between">
99             <div>
100               <p className="text-sm font-semibold text-slate-100">{svc.name}</p>
101               <p className="text-xs text-slate-500">{svc.description}</p>
102             </div>
103             <Badge tone={badgeTone(svc.status)}>{svc.status}</Badge>
104           </div>
105
106           <p className="mt-3 font-mono text-xs text-shield-cyan">{svc.targetProxy}</p>
107           <p className="font-mono text-xs text-slate-400">
108             Rate Limit Rule: {svc.rateLimit.capacity} cap / {svc.rateLimit.refillRate}s
109           </p>
110
111           <div className="mt-2 flex flex-wrap gap-1">
112             {svc.methods.map((method) => (
113               <Badge key={method} tone="neutral">
114                 {method}
115               </Badge>
116             ))}
117           </div>
118
119           <div className="mt-3 flex items-center justify-between border-t border-border pt-3">
120             <span className="font-mono text-xs text-slate-400">
121               {svc.latencyMs} ms &middot; {svc.throughputRpm.toLocaleString()} rpm
122             </span>
123             <div className="flex items-center gap-2">
124               <button

```

```

125         aria-label={svc.status === 'down' ? 'Enable service' : 'Disable service'}
126         onClick={() => toggleService(svc.id)}
127         className="rounded p-1 text-slate-400 hover:text-shield-cyan"
128     >
129         &#9679;
130     </button>
131     <button
132         aria-label="Delete service"
133         onClick={() => removeService(svc.id)}
134         className="rounded p-1 text-slate-400 hover:text-status-danger"
135     >
136         &#128465;
137     </button>
138 </div>
139 </div>
140 </div>
141 )))
142
143     {filtered.length === 0 && (
144         <p className="col-span-full py-8 text-center text-sm text-slate-500">
145             No services match the current filters.
146         </p>
147     )}
148 </div>
149 </Card>
150
151     {isRegisterOpen && <RegisterServiceModal onClose={() => setRegisterOpen(false)} />}
152 </div>
153 );
154 }

```

A.3 — Shared Type Definitions

Excerpt of `api/types.ts` covering the shapes both components above depend on.

`src/api/types.ts` (excerpt)

```

1 // api/types.ts (excerpt relevant to Screens 1 & 2)
2
3 export type ServiceStatus = 'healthy' | 'degraded' | 'down';
4
5 export interface Microservice {
6     id: string;
7     name: string;
8     description: string;
9     targetProxy: string;
10    methods: string[];
11    rateLimit: { capacity: number; refillRate: number };
12    status: ServiceStatus;
13    latencyMs: number;
14    throughputRpm: number;
15 }
16
17 export interface AnomalyIncident {
18     id: string;
19     title: string;
20     severity: 'critical' | 'warning';
21     clientId: string;
22     rule: string;
23     relativeTime: string;
24     action: 'AUTO_BLACKLISTED' | 'FLAGGED_FOR_MONITORING';
25 }
26
27 export interface KpiValue {
28     value: number;
29     delta: number; // signed percentage change vs. previous window
30 }
31
32 export interface TelemetryContextValue {
33     kpis: {
34         throughput: KpiValue;
35         successRate: KpiValue;
36         latencyMs: KpiValue;
37         shieldBlocked: KpiValue;
38     };
39     trafficSeries: Array< { t: string; success: number; clientError: number; serverError: number; blocked: number }>;
40     tokenBucket: {
41         available: number;
42         capacity: number;
43         refillRate: number;
44         drain: (amount: number) => void;
45     };
46     latencyHistogram: { p50: number; p95: number; p99: number };
47     incidents: AnomalyIncident[];
48     services: Microservice[];

```

```
49 toggleService: (id: string) => void;  
50 removeService: (id: string) => void;  
51 refresh: () => void;  
52 }
```